

git

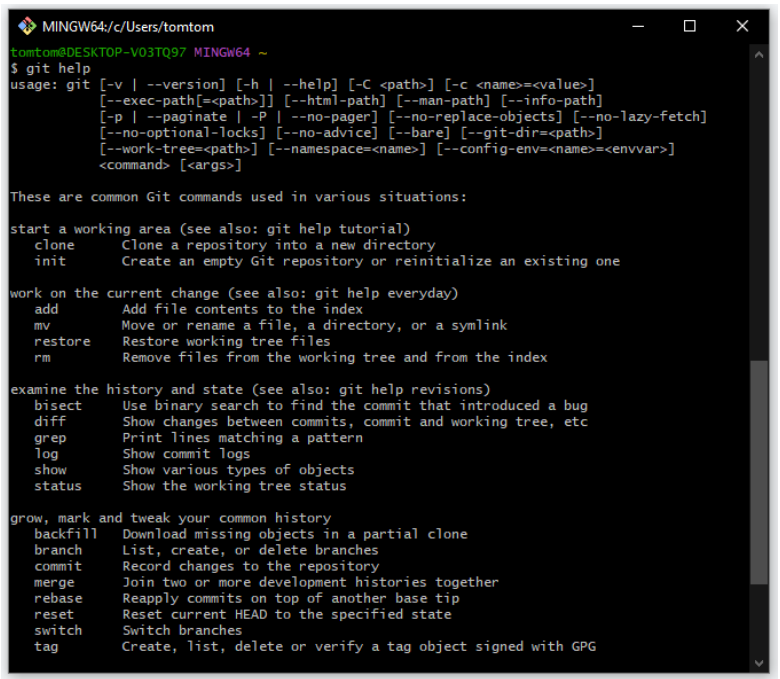
Desarrollo de Software

VCS: Funcionalidades clave

- Rastrea y gestiona los cambios en el código fuente del software.
- Permite el seguimiento individual de cada desarrollador, identificando áreas de conflicto y evitando sobreescrituras.
- Mantiene un historial detallado de toda la vida del proyecto.
- Permite revertir al instante el código a cualquier punto previo en el tiempo.

Qué es Git?

- Git es un Sistema de Control de Versiones (VCS)
- Creado por Linus Torvalds (creador de Linux)
- Es la herramienta más adoptada de su clase; todo desarrollador la conoce.



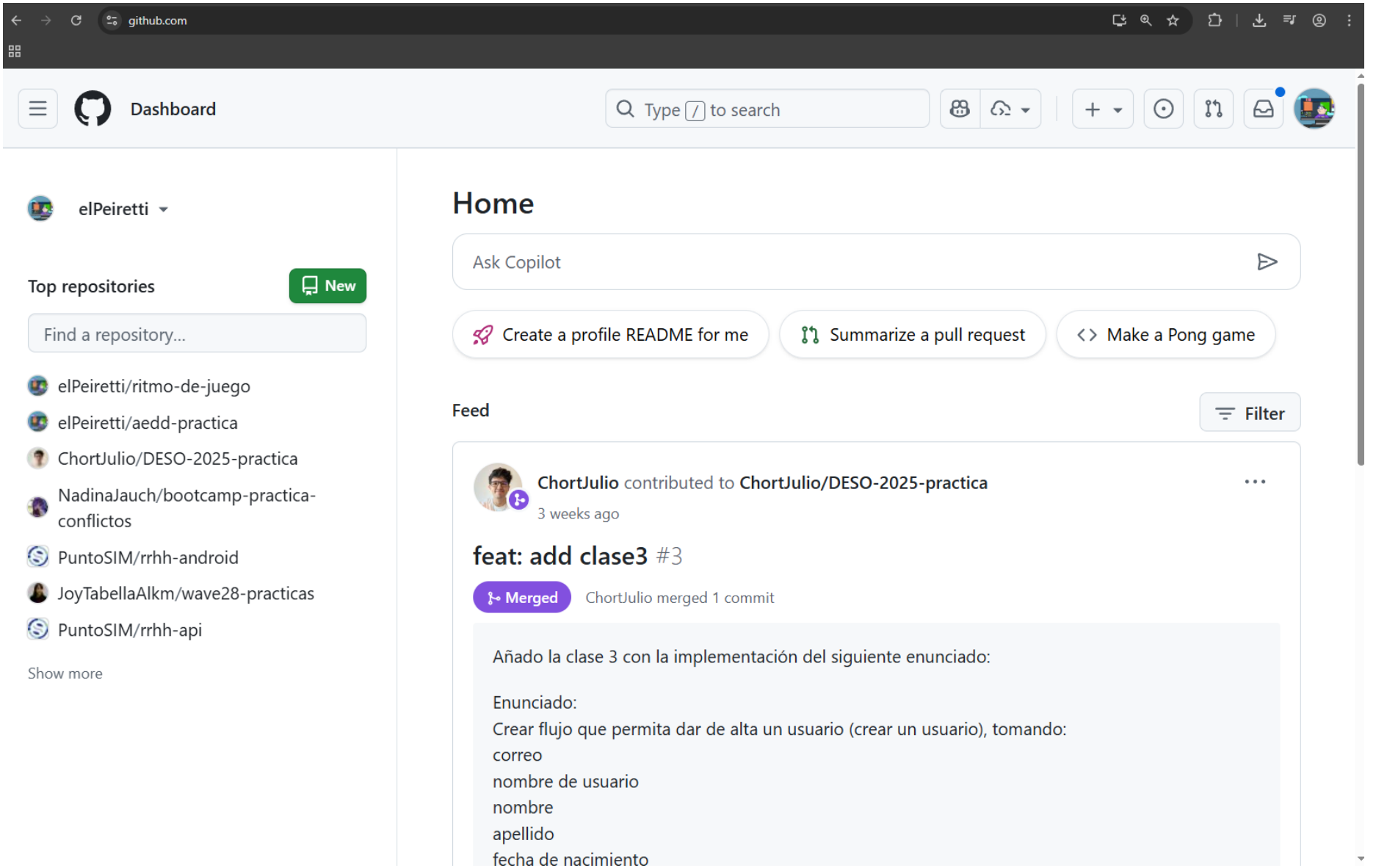


Qué es Github?

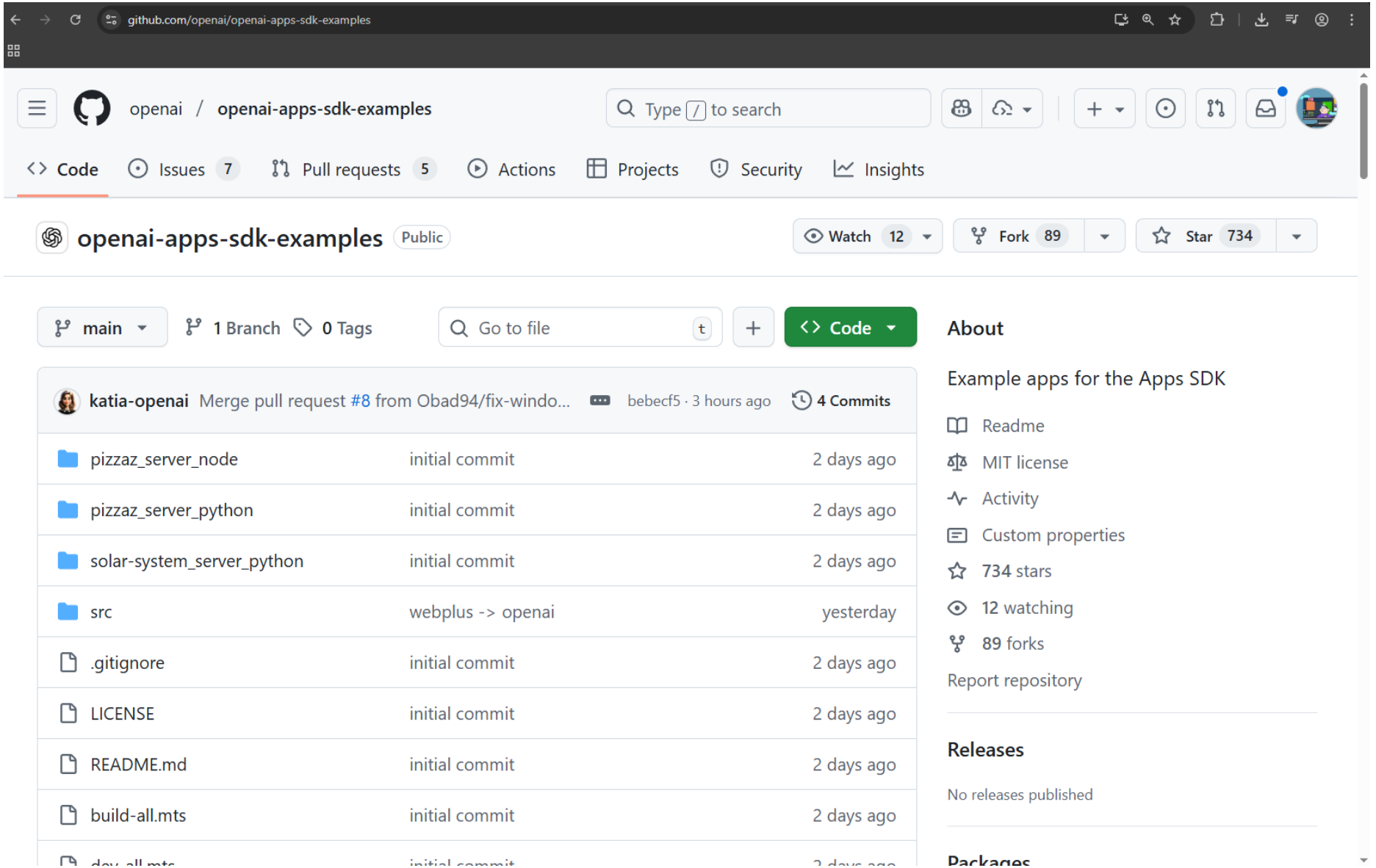
- Es una plataforma en la nube para almacenar, compartir y colaborar en la escritura de código.
  - Rastrea y gestiona los cambios en tu código a lo largo del tiempo.
  - Permite que otros revisen tu código y hagan sugerencias para mejorarlo.
  - Colabora en un proyecto compartido sin preocuparte de impactar el trabajo de los demás antes de estar listo para integrarlo.

https://docs.github.com/en/get-started/start-your-journey/about-github-and-git

Qué es Github?



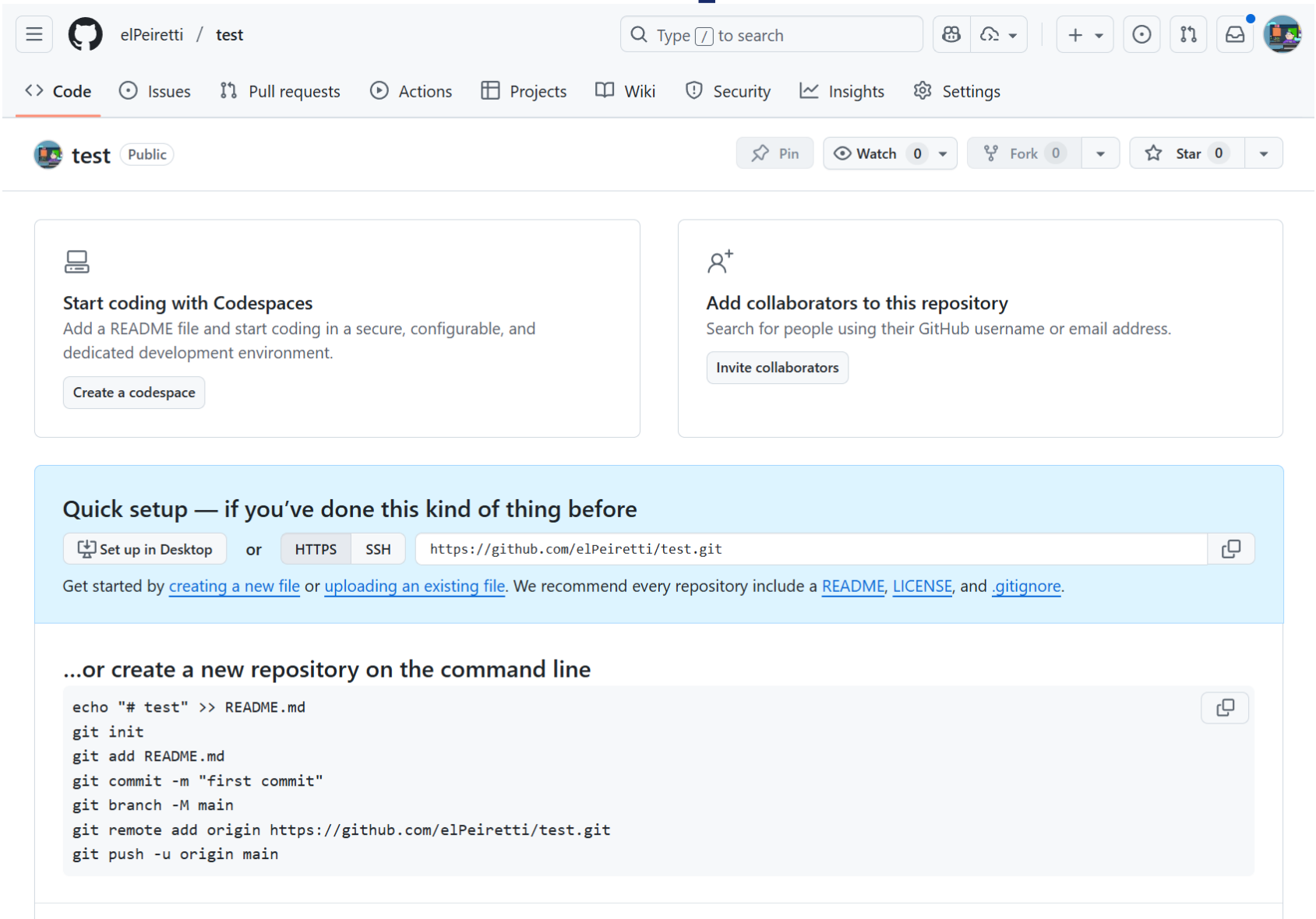
Qué es Github?



Cómo empezar?

1. Instalar git: https://git-scm.com/downloads
2. Crear una cuenta en GitHub: https://github.com/
3. Configurar usuario en git:
   a. Se puede chequear que git esté instalado con el comando git --version
   b. git config --global user.name "John Doe"
   c. git config --global user.email johndoe@example.com
4. Crear un repositorio en Github y seguir los pasos indicados.

Cómo empezar?



Cómo empezar?

Eso es todo

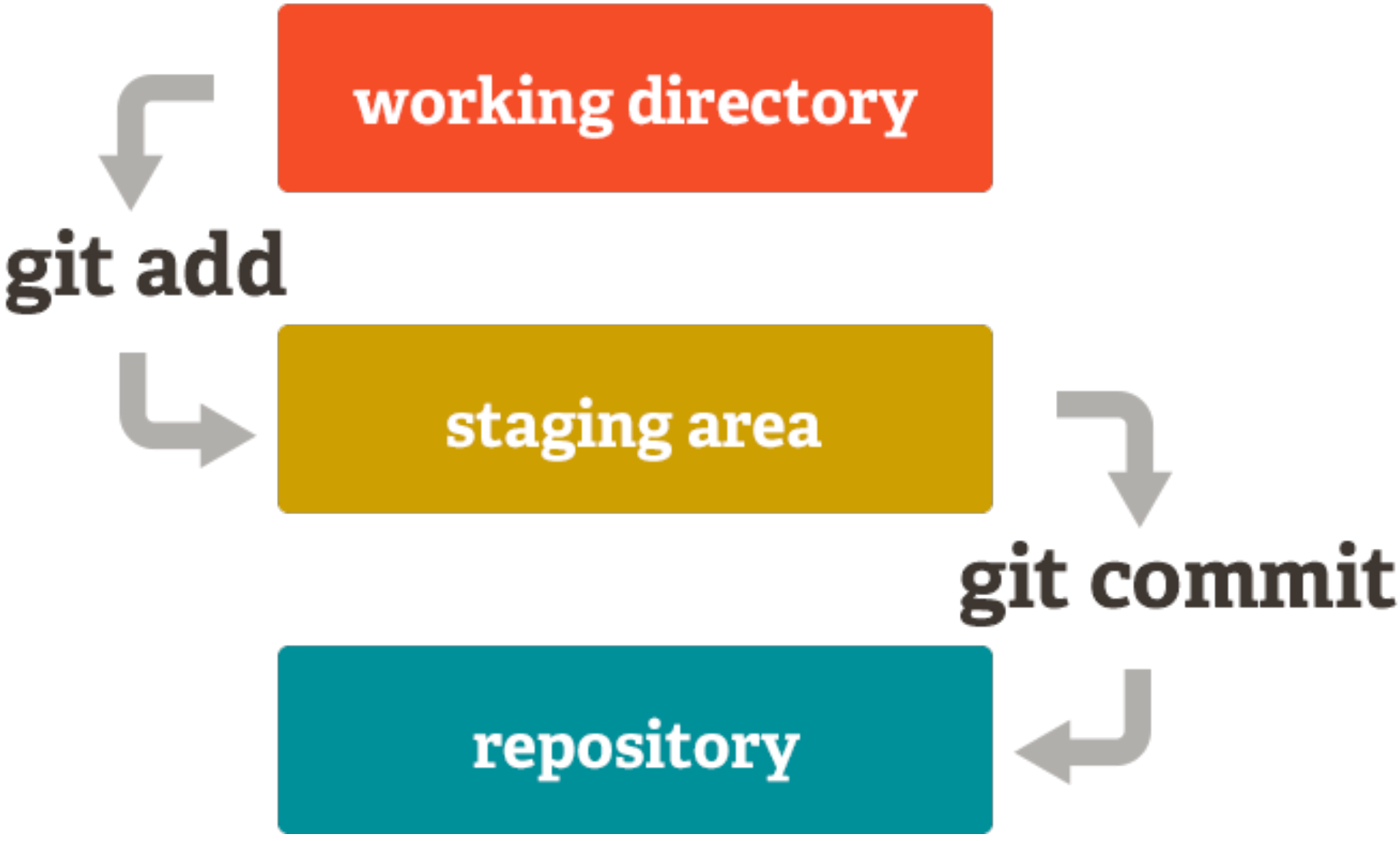


Git: TOP 10 comandos más utilizados

(basados en mi experiencia)

1. git add
2. git commit -m
3. git push
4. git checkout
5. git pull
6. git status
7. git rebase -i
8. git reset
9. git merge
10. git log

Mención honorífica: git branch


working directory
git add
staging area
git commit
repository

Ejemplo



Git: ejercicio - parte 1

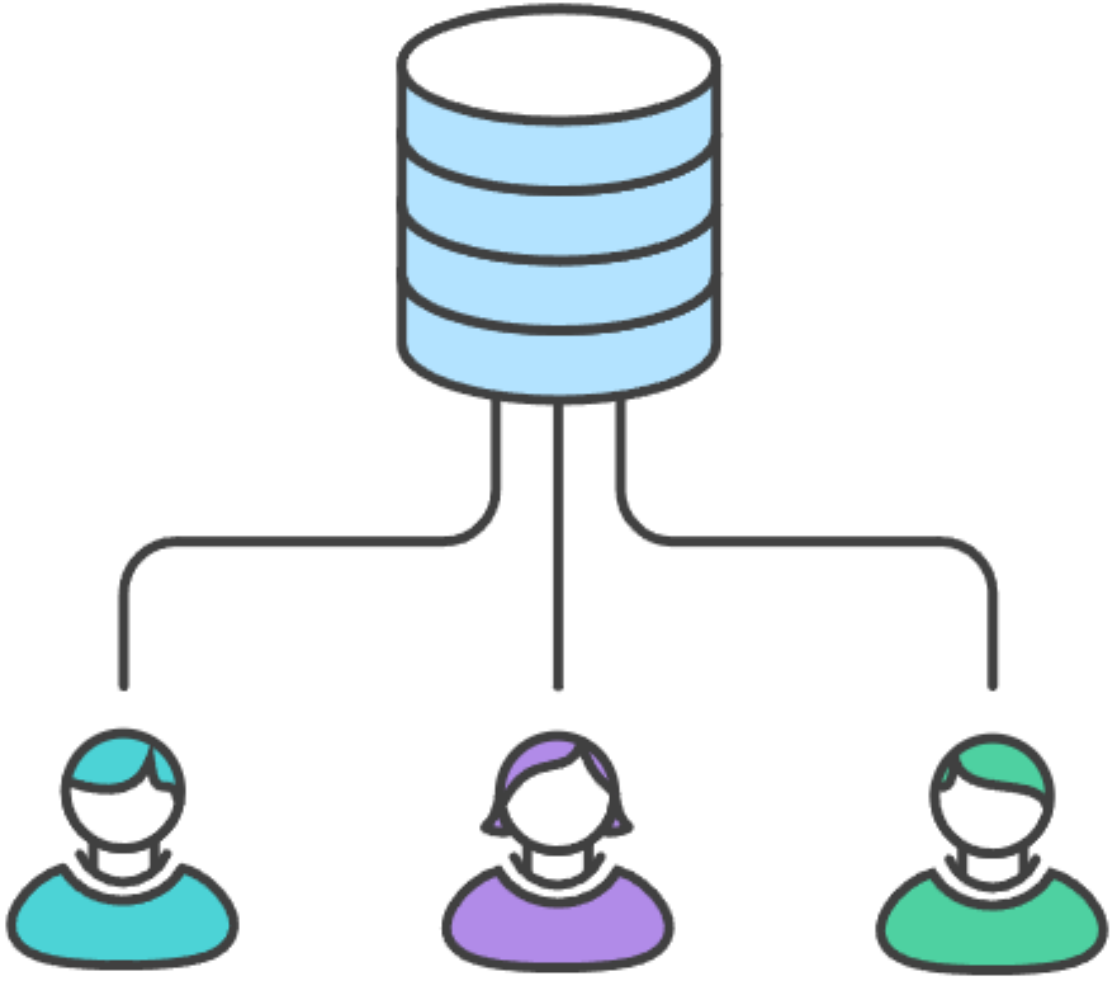
1. Crear un nuevo repositorio siguiendo los pasos anteriores.
2. Crear dos archivos nuevos: `main.java` y `other.txt`.
3. Agregar contenido a los archivos creados.
4. Realizar commit de los cambios y subirlos al repositorio remoto.
5. Editar el archivo `main.java` agregando nuevas líneas.
6. Preparar los cambios (stage), y luego agregar más líneas de texto al mismo archivo.
7. Verificar el estado del repositorio local y observar qué sucedió.
8. Realizar commit de los cambios sin incluir la última modificación y subirlos.
9. Revisar el repositorio de GitHub y observa qué sucedió.
10. Revertir cambios que no fueron stageados.

Git: ejercicio - parte 2

1. Crear una nueva rama y comenzar a trabajar en ella.
2. Editar el archivo `other.txt`.
3. Subir los cambios al repositorio remoto. Consultar el repositorio de GitHub para ver qué cambió.
4. Mergear la nueva rama con main y enviar los cambios.
5. Eliminar la rama creada anteriormente.
6. Consultar el repositorio de GitHub para ver qué cambió.
7. Revertir el merge y subir los cambios a GitHub.

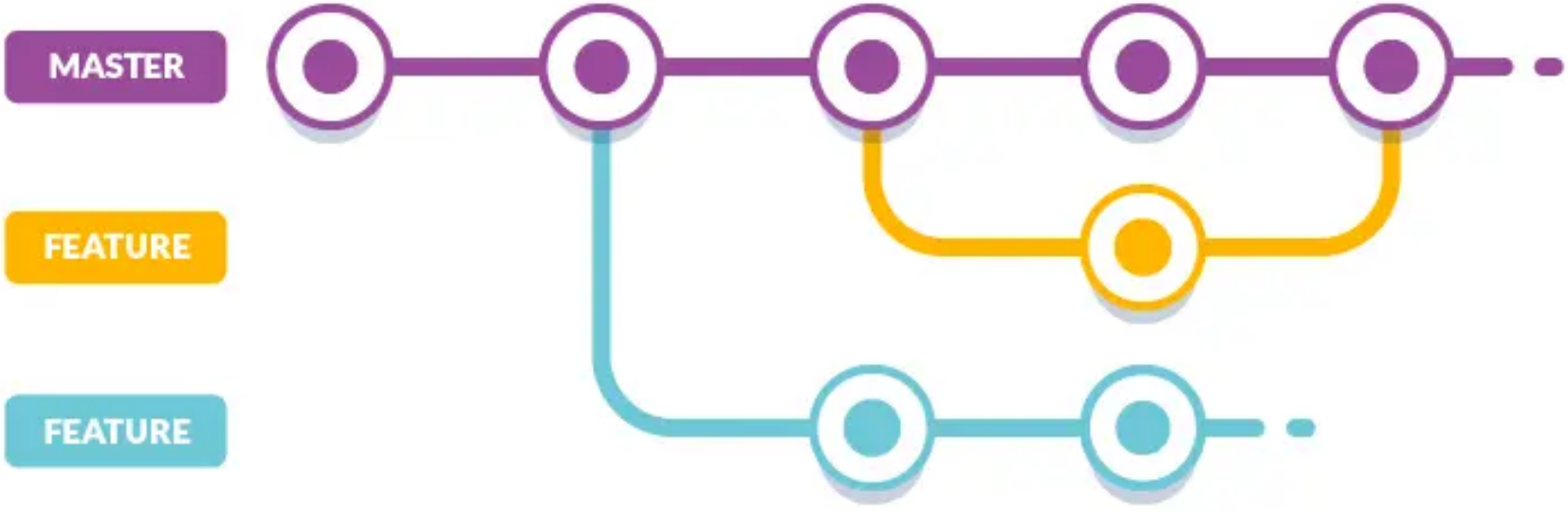
Workflows

Workflows: Centralized



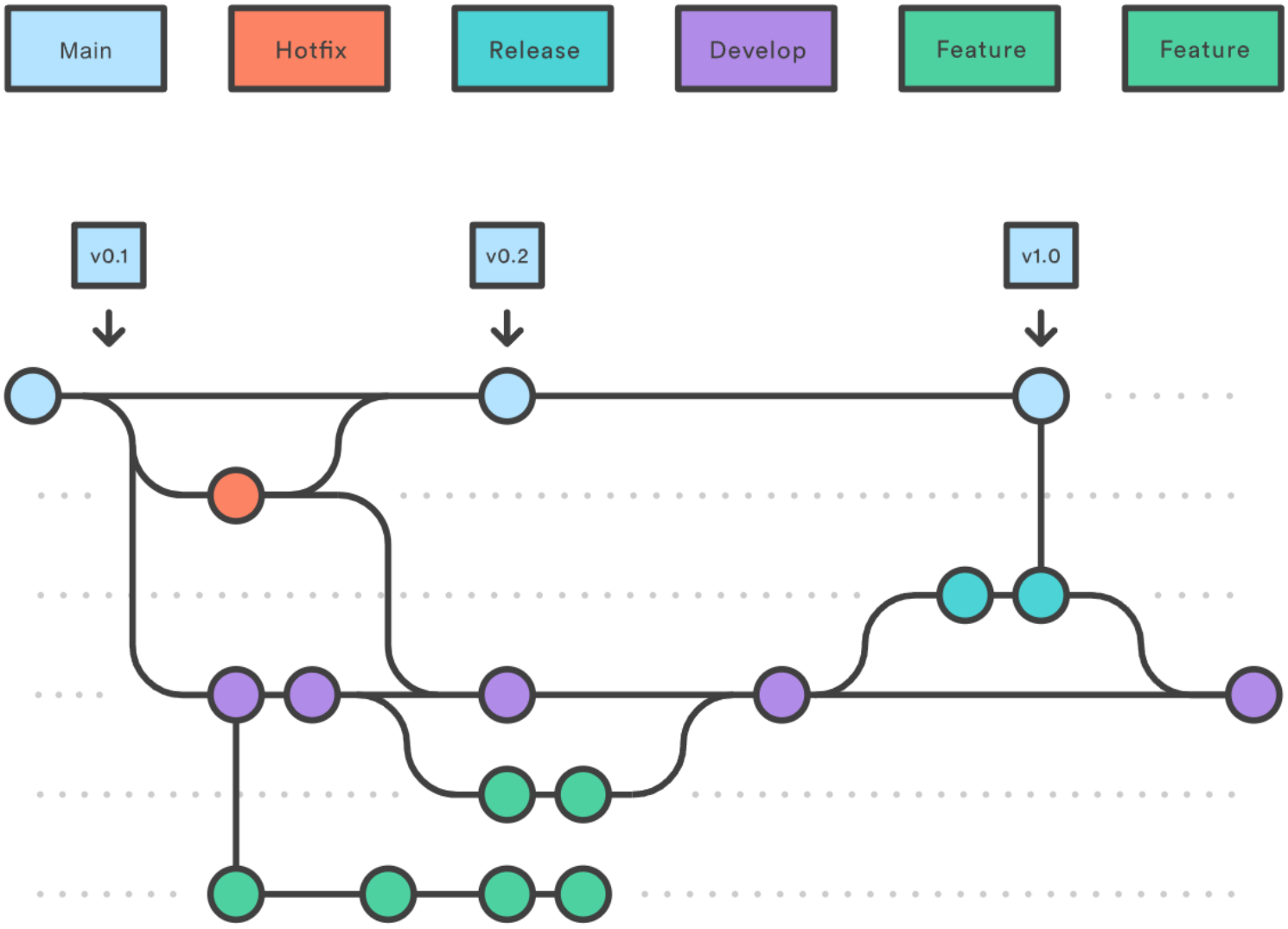
https://www.atlassian.com/git/tutorials/comparing-workflows

Workflows: Feature Branch



https://www.atlassian.com/git/tutorials/comparing-workflows

Workflows: Gitflow



https://www.atlassian.com/git/tutorials/comparing-workflows

Ejemplo

